14 October 2026

The Editor-in-Chief
Computers & Security
Elsevier

Dear Editor,

We are pleased to submit our manuscript, **"Benchmarking Post-Quantum Key Encapsulation in S3-Compatible Object Storage under Client Concurrency"**, for consideration as a research article in *Computers & Security*.

Data encrypted today under classical public-key algorithms could be decrypted once large quantum computers exist. Cloud object stores keep data for years, so the client-side envelope encryption that protects it must move to post-quantum key encapsulation such as ML-KEM (FIPS 203), which adds a key-encapsulation step and a larger ciphertext to every stored object. So far, the cost of ML-KEM has been measured for the algorithm on its own and for securing network connections, but not for encrypting each object in a storage system that serves many clients at once. Practitioners planning this migration therefore have no measurement of whether, when and why it slows their storage path.

Our manuscript addresses this problem. We measured per-object ML-KEM-768 against X25519, RSA-4096, a symmetric-only baseline and two hybrid constructions on an S3-compatible object store, with a Python client on x86 and ARM and a native Go reader, at 1 to 500 concurrent workers. The study comprises an exploratory campaign (1,970 runs), a pre-registered confirmatory campaign (1,495 runs, 97.6 million requests) analysed with paired equivalence tests and Holm correction, and pre-registered follow-up controls (480 runs with the Python client and 90 with the native reader). The main findings are:

- ML-KEM lowered throughput only where the client was at or near CPU saturation, with one unexplained exception that we report.
- In the saturated pre-registered cell the Python client was 4.0 % (x86) and 4.9 % (ARM) slower than with X25519. Controls on x86 traced this deficit mainly to the library binding, and with a single foreign call ML-KEM was equivalent to X25519 within ±2 %.
- A native reader saturated on one core was 9.5 % slower, showing that the size of the cost depends on the implementation.
- The cost every implementation pays is storage: 1,056 more bytes of ciphertext per object than X25519.

We believe the work suits *Computers & Security* because it gives security engineers and storage operators planning a post-quantum migration a measured basis for that decision, together with a simple procedure (comparing client CPU per request under saturation) that they can apply to their own systems. The paper states its limits openly: it covers two clients, a single-node store and one cloud provider, and it lists the observations that remain unexplained.

We confirm that this manuscript is original, has not been published previously, and is not under consideration for publication elsewhere. All authors have read and approved the manuscript and agree with its submission to *Computers & Security*. The declarations of competing interests, funding and the use of generative AI are included in the manuscript. Supplementary material describing the open-loop capacity of the store is provided as a separate file. The code, configurations, pre-registration records and per-run results are publicly available at https://github.com/revanthreddy0906/pqc-object-store.

Thank you for considering our submission. We look forward to hearing from you.

Yours sincerely,

Revanth Reddy Lingala (corresponding author)
on behalf of all authors: K. Gangadhara Rao, Padigela Srujan, Saarthak Manocha and Vainavi Venkat
Department of Information Technology, Chaitanya Bharathi Institute of Technology, Hyderabad, Telangana, India
revanthreddy0906@gmail.com